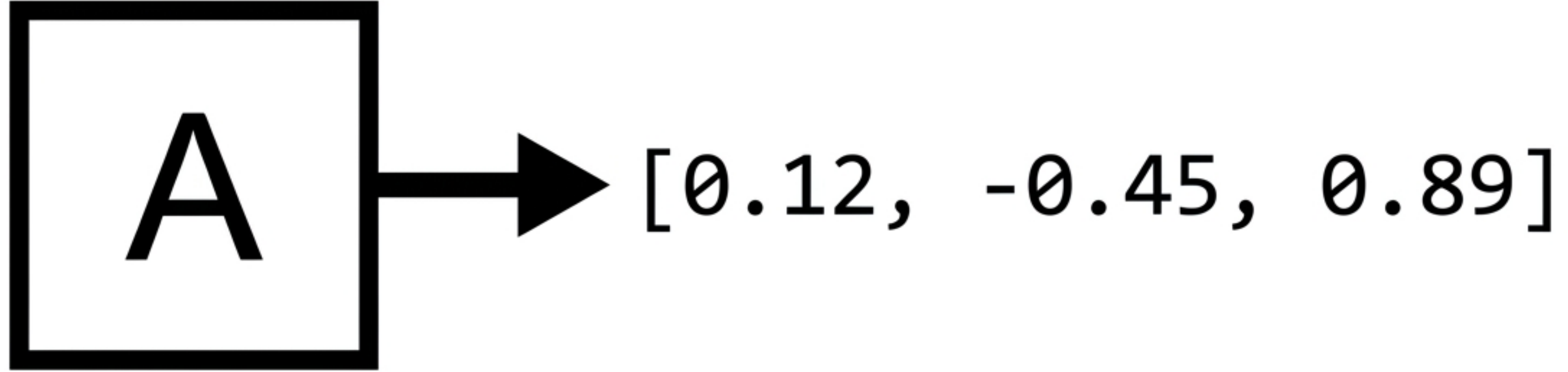




Preparing Text for Large Language Models

The Data Pipeline from Raw Words to Continuous Vectors.

NEURAL NETWORKS CANNOT READ

Large Language Models process **continuous-valued mathematical vectors**, not **discrete, categorical text**. The **fundamental challenge of data preparation** is translating human language into a **continuous vector** space compatible with backpropagation and matrix multiplication.

THE PROBLEM

~~APPLE~~

THE SOLUTION

$$\begin{bmatrix} 0.12 \\ -0.45 \\ 0.89 \end{bmatrix}$$


The Data Preparation Blueprint

Converting human text into a GPT-compatible tensor requires a precise 5-stage assembly line. This pipeline breaks down raw inputs, maps them to integers, structures them for training, and mathematically projects them into a continuous space.



Step 1: Slicing Text into Discrete Tokens

Text must be split into individual tokens—words, subwords, or punctuation. Capitalization and punctuation are preserved as distinct tokens, providing critical syntactical context that the model must learn.

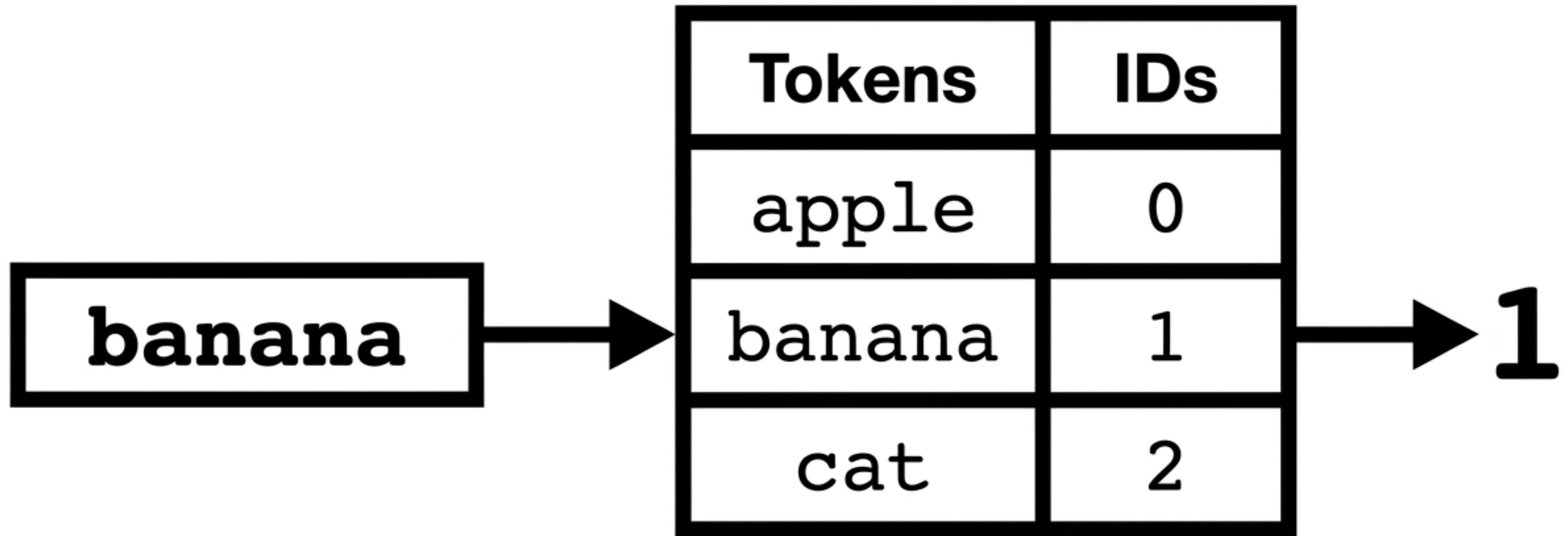
Hello, world!



[Hello, world!]

Step 2: Mapping Tokens to Integers

To convert strings to numbers, we build a vocabulary by sorting all unique tokens alphabetically and assigning a unique integer (**Token ID**) to each. An inverse vocabulary allows the model to decode numbers back into human text.



Handling Unknowns and Boundaries

Special tokens provide structural context to the LLM:

<|unk|>

Unknown Word Marker: Replaces out-of-vocabulary words to prevent basic tokenizers from crashing.

<|endoftext|>

Boundary Marker: Acts as a definitive boundary marker when concatenating multiple unrelated documents. It ensures the model does not hallucinate false connections between independent texts.

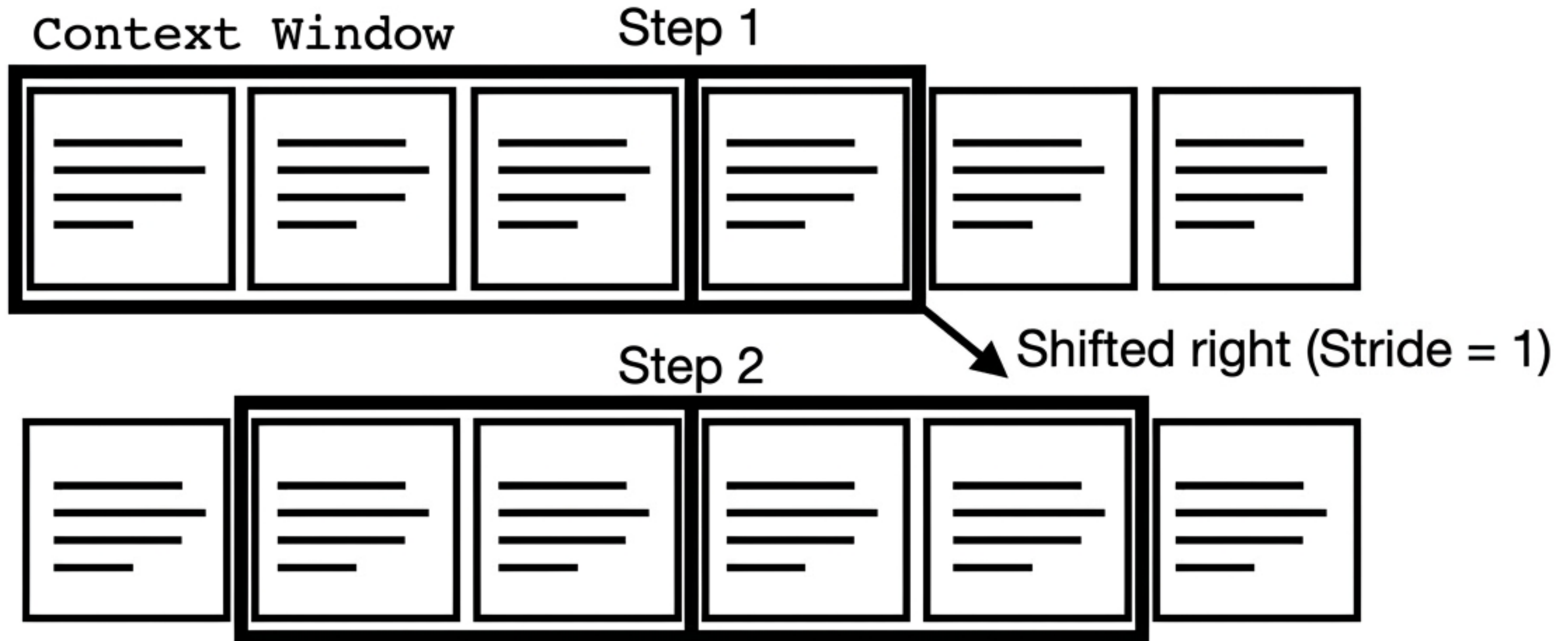
Regex vs. Byte Pair Encoding (BPE)

Modern GPT models use Byte Pair Encoding. Because BPE can break any unknown word down into individual subwords or characters, it guarantees that the model can process any text without ever needing an `<|unk|>` token.

Feature	Simple Regex	Byte Pair Encoding (BPE)
Mechanism	Splits by whitespace/punctuation.	Iteratively merges frequent characters.
Unknown Words	Fails; requires <code>< unk ></code> token.	Breaks unknown words into subwords/characters.
Vocabulary Size	Variable	Fixed at exactly 50,257 (for GPT).

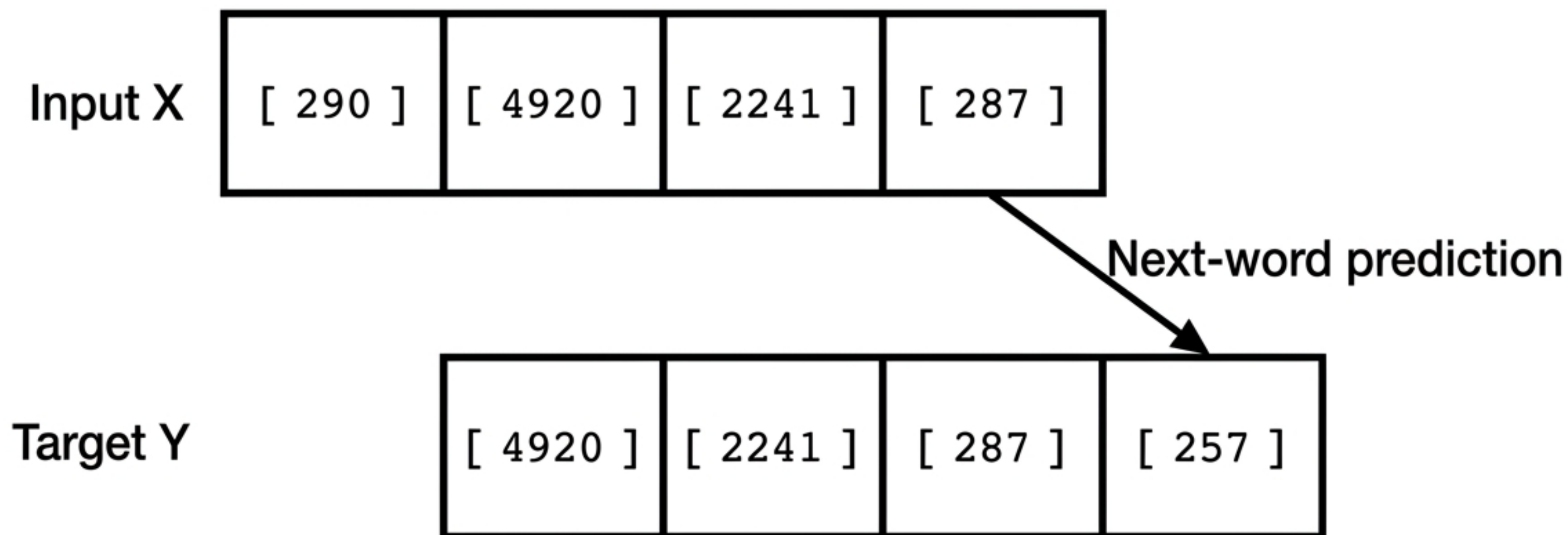
Step 3: Extracting Data via Sliding Window

LLMs are trained on a 'next-word prediction' task. To generate vast amounts of training data from a single document, a sliding window moves across the text sequence to create overlapping context chunks.



Creating Inputs (X) and Targets (Y)

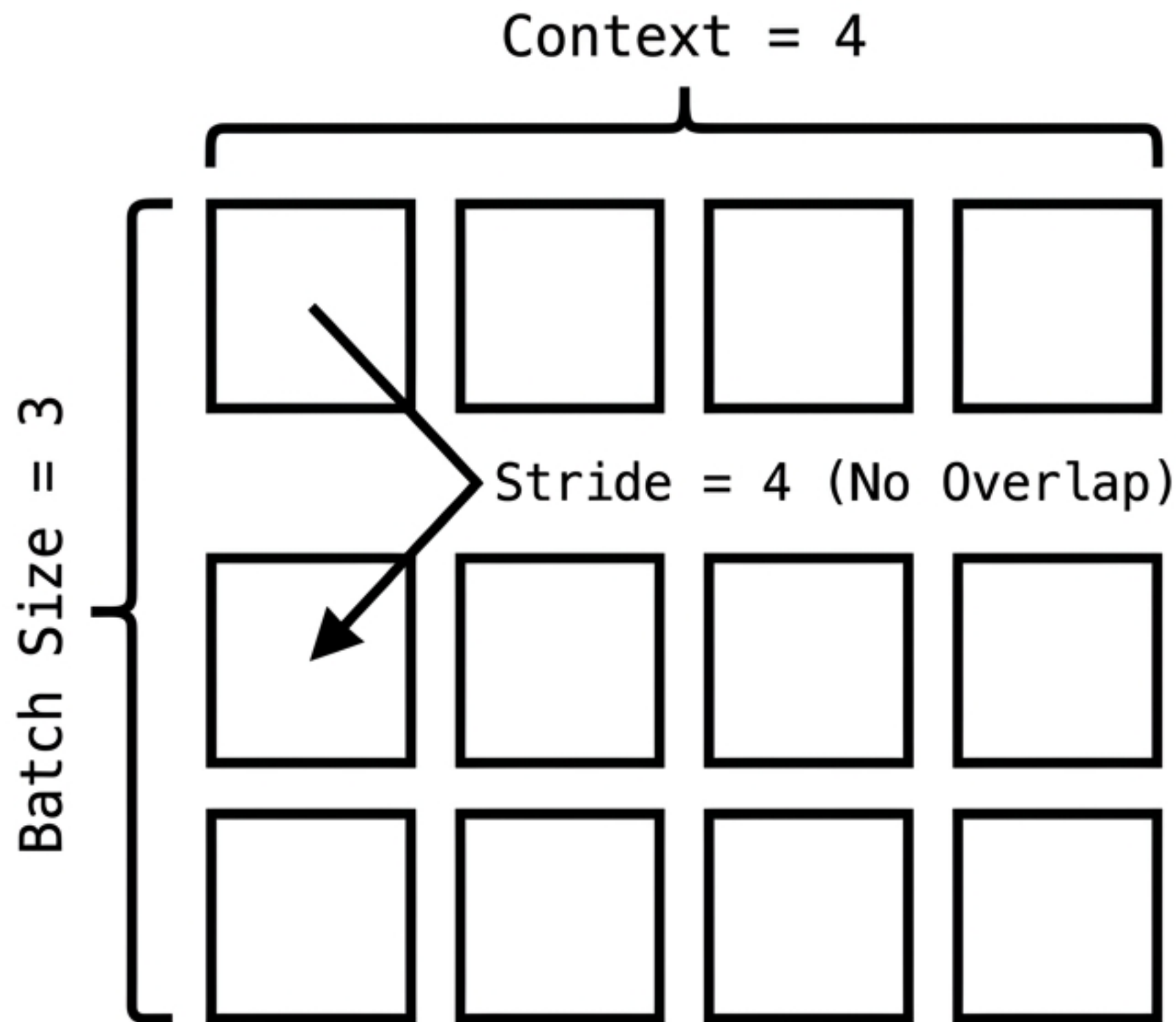
For every input sequence (X), the target sequence (Y) is the exact same input sequence shifted right by one position. The LLM's goal during training is to successfully predict that final shifted target token.



Batches and Strides

Data is fed into the LLM in multidimensional arrays (tensors).

- Batch Size: Dictates how many sequences are processed simultaneously.
- Stride: Dictates the slide distance between samples. Setting the the stride equal to the context length prevents overlapping text, reducing the risk of overfitting.



Step 4: The Token Embedding Lookup

A Token ID is merely a discrete integer. The Embedding Layer acts as a lookup table, transforming that integer into a continuous vector (spanning up to 12,288 dimensions in GPT-3). These matrix weights are dynamically updated and optimized during model training.

Embedding Layer Weights

-0.51	-0.67	-0.82	-0.23	0.01	-0.27	-0.03	-0.64
-2.82	-0.51	-1.91	-0.43	0.05	-0.49	-0.26	-1.07
-1.38	-0.57	-1.53	-0.39	0.01	-1.43	-0.93	-2.38
-0.97	-0.82	-0.31	-0.19	0.08	-0.73	-0.67	-0.96
-3.85	-0.82	-0.68	-0.10	0.23	-0.08	-0.73	-1.47
-2.84	-0.78	-1.40	-0.15	0.09	-2.84	-0.78	-1.40
-1.84	-0.60	-0.12	-0.43	0.04	-1.29	-0.24	-0.96
-1.32	-0.29	-0.12	-0.20	0.07	-1.93	-0.78	-0.97
-1.86	-0.73	-0.16	-0.54	0.03	-1.26	-0.69	-1.55
-1.52	-0.85	-0.40	-0.52	0.03	-1.23	-0.62	-0.93

Token ID

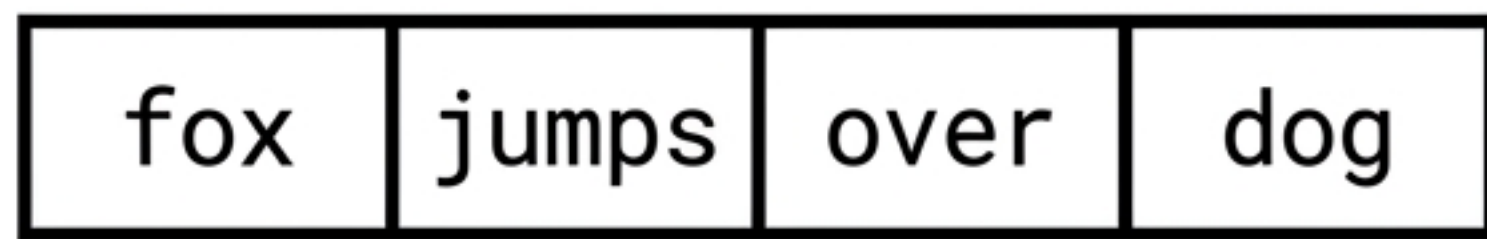
[5]

Continuous Vector

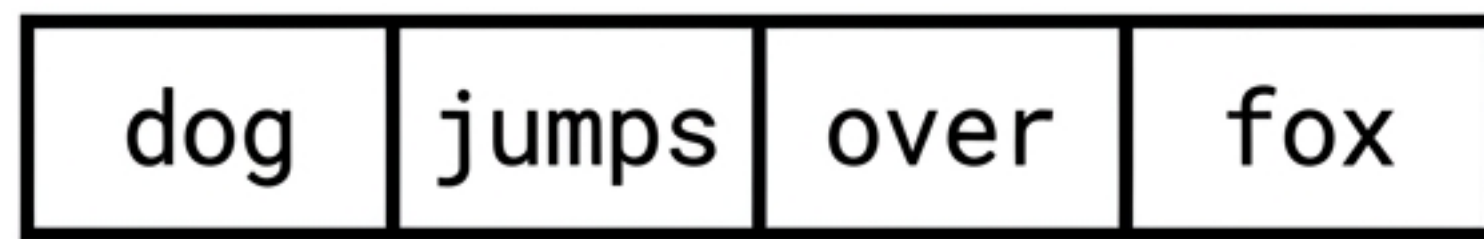
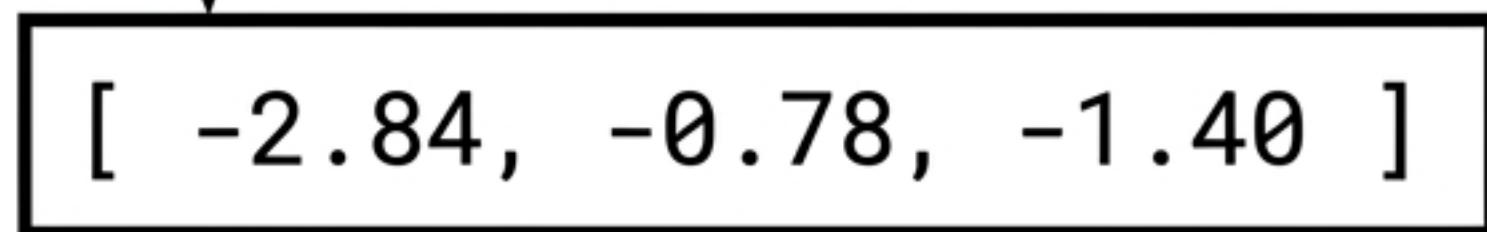
[-2.84, -0.78, -1.40]

The Blind Spot of Self-Attention

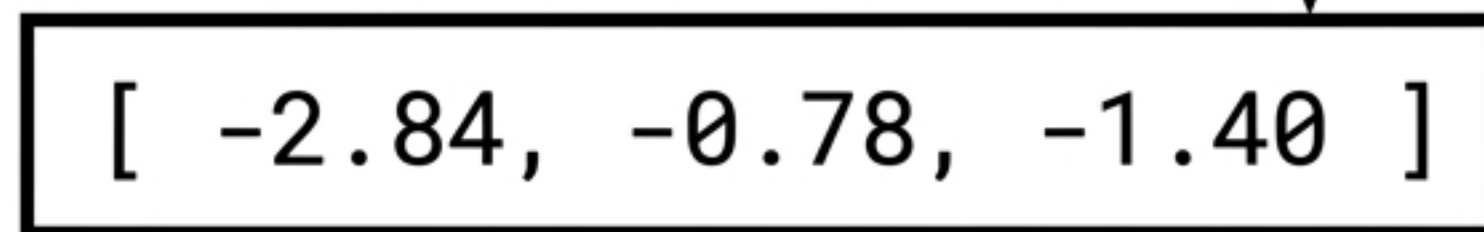
The core mechanism of an LLM has no inherent concept of sequence or order. Because the embedding lookup is deterministic, the model sees “fox jumps” and “jumps fox” as identical sets of vectors unless we explicitly inject spatial awareness.



ID: 5



ID: 5



=

Step 5: Injecting Order with Positional Embeddings

Absolute positional embeddings are created as a separate set of continuous vectors that map to exact sequence positions (0, 1, 2...). These are added directly to the token embeddings. The resulting vectors encode both the meaning of the word and its exact location in the sentence.

[Token Embedding Vector]

+

[Positional Embedding Vector]

=

[Final Input Vector]

Synthesis: The Complete Input Pipeline

The journey from word to tensor is complete. Raw text is tokenized via BPE, mapped to unique integers, chunked into batches via a sliding window, converted to vectors via a lookup weight matrix, and stamped with positional context. Only now can the data enter the LLM's transformer blocks.

